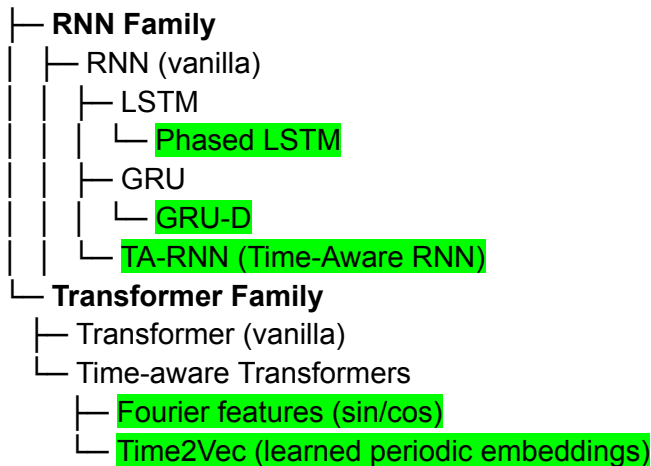


Recurrent Neural Networks

- General Idea: RNNs are a type of neural network designed to process sequences of data by using an internal “memory”
- RNNs typically implicitly use “time” by assuming that some unspecified amount of time has passed in between each step
 - Must use a modified version of an RNN (T-RNN) or another machine learning methods

Hierarchy - Highlighted are variants that are being considered

Neural Networks



Time-Aware Recurrent Neural Networks

- A variation of RNN designed for irregular time series.
- Modifies the hidden state update by incorporating the time gap (Δt) since the last observation.
- Uses a decay function so that information from the past fades more strongly as the time gap increases.
- General Idea
 - More recent events -> higher impact
 - Less recent events -> lower impact
- **Pros/Cons**
 - **Pros**
 - Conceptually simple, easier to understand than other models
 - Good first step for handling irregular sequences without lots of complexity
 - **Cons**
 - Doesn't attempt to fix vanishing gradient issues like LSTM/GRU do
 - Less powerful for long-term dependencies
 - May underperform compared to GRU-D/Phased LSTM on complex data

Gated Recurrent Unit (GRU)

- GRU is a type of simpler RNN that uses update/reset gates to control information flow
 - In other words, it uses gates to decide what information to keep/forget as data moves through a sequence

Gated Recurrent Unit with Decay (GRU-D)

- Modified version of Gated Recurrent Unit (GRU)
- Specifically designed for multivariate time series analysis to handle:
 - Missing values
 - Irregular sampling (not evenly spaced in time)
- Adds a “decay function” so that:
 - Old or missing information gradually fades over time
 - More recent data has more influence on predictions
- Handles missing data by replacing it with a decayed estimate rather than just dropping or guessing it
- **Pros/Cons**
 - **Pros**
 - Specifically designed for irregular sampling/missing values
 - Simpler and faster than LSTM variants (since it's GRU-based and GRU is simpler than LSTM)
 - **Cons**
 - The “decay” approach isn't always the right way to approach every problem
 - Doesn't always fit real dynamics
 - May struggle if the irregular timing carries complex patterns beyond simple fading
 - Perhaps this means it's bad for our use case?

Long Short-Term Memory (LSTM)

- A type of RNN that is specifically designed to handle long sequences of data
 - Regular RNNs try to remember everything step by step but often “forget” long-term details (another form of the vanishing gradient problem)
 - LSTMs fix this by adding a sort of “memory notebook”
 - Uses gates to decide what happens to information
 - Forget gate: What old info should be erased
 - Input gate: What new info to write down
 - Output gate: What part of the memory to share now
 - Analogy:
 - When reading a story, you don’t want to forget important details from earlier (e.g. who the main character is) but you also don’t want to remember every single word
 - TLDR: An LSTM is just an RNN that’s more particular about what it remembers and forgets, allowing it to capture both long-term and short-term patterns

Phased Long Short-Term Memory

- Normal LSTMs expect input at every step (like every second or every word)
- Phased LSTMs add a time gate (a clock)
 - The clock tells the LSTM when to “wake up” and process an input
- This change makes Phased LSTMs much better for irregular or event-based data
 - Event-Based Data: Data that is only recorded when something happens, not on a fixed schedule
 - E.g. a security camera that only saves a frame when it detects motion
- **Pros/Cons**
 - **Pros**
 - Efficient (only updates during “open” phases)
 - Great for event-driven or sparse data
 - **Cons**
 - Needs careful tuning of the clock parameters
 - If events don’t align well with the clock, performance can suffer
 - More complex to implement and train compared to regular LSTMs
 - Less common than GRU/LSTM, so there are fewer libraries/tutorials available

Transformers

- A type of neural network that's great at finding patterns in sequences
- Doesn't read data step by step (like RNNs do), instead it looks at the sequence as a whole
- Uses "attention"
 - Attention -> The model deciding which parts of the sequence are important to pay attention to when making a decision
 - E.g. for the sentence "The cat sat on the mat", attention would determine that the word "cat" is strongly connected to the word "sat"
- Don't naturally understand order, just see a group of numbers

Transformers with Time Embeddings

- A time embedding is a way of encoding when each observation happened (and feeding it into the transformer along with the flux value)
- Multiple variations
 - Positional encoding: Gives each step a unique "position code" (step 1, 2, 3, etc)
 - Works well for evenly spaced data, perhaps not a good choice for our use case
 - Relative Time Encoding: Tells the model the actual gap between measurements
 - Time2Vec/Fourier Features: Turns time into sine/cosine waves so the model can easily learn periods
 - Time2Vec/Fourier are similar in concept but different in implementation
 - Fourier uses sinusoids
 - Simple, fast, work well if periods are known
 - Time2Vec is more flexible; attempts to find a linear term and a periodic term to represent the data
 - Better for messy data where periods differ
 - **Probably the better approach for our use case**

My Conclusions

- Phased LSTMs/Time2Vec Transformers appear to be the best approaches
- GRU-D and TA-RNN will likely be too simple for the complex patterns we want to observe
- Phased LSTMs:
 - More complicated in setting parameters
 - Limited Library availability, extra (heavy) work needs to be done
- Time2Vec Transformers
 - Much simpler parameter setting
 - Limited library availability, extra (light) work needs to be done
- Time2Vec stands out above the rest in my opinion